UNIVERSIDAD MARIANO GÁLVEZ

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Proyecto Final - Métodos Numéricos con Python

DATOS GENERALES

Prerrequisito(s): 70 créditos
Código del Curso: 021
Nombre del Curso: Métodos Numéricos - Python
Valor del proyecto: 15 puntos sobre nota del examen final

DESCRIPCIÓN DEL PROYECTO

El proyecto final consiste en el desarrollo de una aplicación en Python que implemente los métodos numéricos estudiados durante el curso. La aplicación debe permitir al usuario seleccionar distintos métodos numéricos y ejecutarlos con parámetros personalizados, visualizando los resultados obtenidos de manera gráfica y numérica.

Este proyecto integra los conocimientos adquiridos en el curso, desde la programación en Python hasta la implementación de algoritmos numéricos específicos para resolver problemas matemáticos y de ingeniería.

OBJETIVOS

1. Implementar los principales métodos numéricos estudiados en el curso utilizando Python
2. Desarrollar una interfaz de usuario funcional que permita la interacción con estos métodos
3. Aplicar correctamente las bibliotecas NumPy, SciPy, Matplotlib y SymPy para el desarrollo de soluciones numéricas
4. Documentar adecuadamente el código y el funcionamiento de la aplicación
5. Demostrar comprensión y dominio de los conceptos fundamentales de métodos numéricos

REQUISITOS FUNCIONALES

La aplicación debe implementar los siguientes métodos numéricos, organizados por módulos:

Módulo 1: Cálculo de Raíces

- Método de Bisección
- Método de Newton-Raphson
- Método de la Secante

Módulo 2: Interpolación y Ajuste de Curvas

- Interpolación con el polinomio de Lagrange
- Interpolación con el polinomio de Newton
- Visualización de los resultados de interpolación

Módulo 3: Sistemas de Ecuaciones

- Resolución de sistemas lineales mediante método de Gauss-Seidel
- Factorización LU

Módulo 4: Derivación e Integración Numérica

- Aproximación de derivadas mediante diferencias finitas
- Integración numérica mediante métodos de Trapecio y Simpson

Módulo 5: Ecuaciones Diferenciales

- Resolución de ecuaciones diferenciales ordinarias mediante el método de Euler
- Resolución de ecuaciones diferenciales ordinarias mediante el método de Runge-Kutta

REQUISITOS TÉCNICOS

Lenguaje y Bibliotecas

- Programación en Python 3.x
- Uso obligatorio de las bibliotecas:
    - NumPy: para operaciones numéricas y manipulación de arrays
    - Matplotlib: para visualización de resultados
    - SciPy (opcional): para verificación de resultados
    - SymPy (opcional): para manipulación simbólica

Interfaz de Usuario

- Desarrollar una interfaz gráfica utilizando alguna de las siguientes opciones:
    - Tkinter (incluido en Python)
    - PyQt
    - Streamlit
    - Jupyter Notebook interactivo

Estructura del Proyecto

- Organización modular del código
- Separación clara entre la lógica del programa (implementación de métodos) y la interfaz de usuario
- Implementación de clases y funciones bien definidas
- Manejo adecuado de errores y validaciones de entrada

ENTREGABLES

1. **Código fuente completo**:
    - Archivos `.py` organizados en directorios según módulos
    - Comentarios explicativos en el código
    - Instrucciones de instalación y ejecución
2. **Manual de Usuario**:
    - Documento en formato PDF que explique:
        - Proceso de instalación
        - Instrucciones de uso
        - Descripción de cada módulo y función
        - Ejemplos de uso con capturas de pantalla
        - Casos de prueba recomendados
3. **Manual Técnico**:
    - Documento en formato PDF que incluya:
        - Arquitectura del sistema
        - Diagramas de flujo o UML cuando sea pertinente
        - Explicación de los algoritmos implementados
        - Decisiones de diseño
        - Problemas encontrados y soluciones aplicadas
        - Referencias bibliográficas utilizadas
4. **Presentación**:
    - Diapositivas en formato PDF o PowerPoint
    - Video demostrativo (opcional, máximo 5 minutos)
5. **Archivo ejecutable** (opcional):
    - Versión compilada o empaquetada de la aplicación

CRITERIOS DE EVALUACIÓN (15 PUNTOS)

1. **Implementación correcta de los métodos (6 puntos)**:
    - Precisión y exactitud de los resultados (2 puntos)
    - Eficiencia del código (2 puntos)
    - Cobertura de todos los métodos requeridos (2 puntos)
2. **Interfaz de usuario (3 puntos)**:
    - Diseño intuitivo y facilidad de uso (1 punto)
    - Visualización adecuada de resultados (1 punto)
    - Manejo de errores y validaciones (1 punto)
3. **Documentación (4 puntos)**:
    - Manual de usuario completo y claro (1.5 puntos)
    - Manual técnico detallado (1.5 puntos)
    - Comentarios y documentación interna del código (1 punto)
4. **Creatividad y valor agregado (2 puntos)**:
    - Funcionalidades adicionales (1 punto)
    - Presentación profesional y atención al detalle (1 punto)

FORMATO DE ENTREGA

- Los entregables deberán ser enviados a través de la plataforma del curso
- El código fuente debe estar comprimido en formato `.zip`
- Los manuales y presentación deben estar en formato PDF
- Nombrar los archivos de la siguiente manera:
    - `[Carné]_ProyectoFinalMN_Codigo.zip`
    - `[Carné]_ProyectoFinalMN_ManualUsuario.pdf`
    - `[Carné]_ProyectoFinalMN_ManualTecnico.pdf`
    - `[Carné]_ProyectoFinalMN_Presentacion.pdf`

ANEXO: GUÍA PARA ELABORAR EL PROYECTO

1. Planificación del Proyecto

Antes de comenzar a programar, se recomienda:

- Analizar cada método numérico y comprender su algoritmo
- Diseñar la estructura del proyecto (módulos, clases, funciones)
- Planificar la interfaz de usuario y la experiencia de usuario
- Identificar posibles puntos críticos o desafíos técnicos

2. Desarrollo Iterativo

Se sugiere un enfoque iterativo:

1. Implementar primero los algoritmos básicos sin interfaz (pruebas en consola)
2. Verificar cada método con ejemplos conocidos
3. Desarrollar la interfaz de usuario
4. Integrar algoritmos con la interfaz
5. Realizar pruebas exhaustivas

3. Sugerencias para la Implementación

3.1 Módulo de Cálculo de Raíces

```
def biseccion(f, a, b, tol=1e-6, max_iter=100):
    """
    Método de bisección para encontrar raíces de funciones.

    Parámetros:
    f -- función para la cual se busca la raíz
    a, b -- intervalo inicial [a, b]
    tol -- tolerancia del error
    max_iter -- número máximo de iteraciones

    Retorna:
    raiz -- aproximación de la raíz
    iter -- número de iteraciones realizadas
    error -- error estimado
    historial -- registro de valores intermedios
    """
    # Implementación
    ...
```

3.2 Módulo de Interpolación

```
def lagrange(x, y, x_interp):
    """
    Interpolación mediante el polinomio de Lagrange.

    Parámetros:
    x -- puntos x conocidos
    y -- valores y conocidos
```

```
    x_interp -- puntos x a interpolar

    Retorna:
    y_interp -- valores interpolados
    """
    # Implementación
    ...
```

3.3 Interfaz de Usuario

Para una interfaz con Tkinter:

```
import tkinter as tk
from tkinter import ttk
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

class AplicacionMetodosNumericos:
    def __init__(self, root):
        self.root = root
        self.root.title("Métodos Numéricos - Proyecto Final")

        # Crear pestañas para los diferentes módulos
        self.notebook = ttk.Notebook(root)
        self.tab_raices = ttk.Frame(self.notebook)
        self.tab_interpolacion = ttk.Frame(self.notebook)
        # ...más pestañas

        self.notebook.add(self.tab_raices, text="Cálculo de Raíces")
        self.notebook.add(self.tab_interpolacion, text="Interpolación")
        # ...agregar más pestañas

        self.notebook.pack(expand=1, fill="both")

        # Configurar cada pestaña
        self._configurar_tab_raices()
        self._configurar_tab_interpolacion()
        # ...configurar más pestañas

    def _configurar_tab_raices(self):
        # Elementos de la interfaz para cálculo de raíces
        ...

    def _ejecutar_biseccion(self):
        # Obtener valores de la interfaz
        # Ejecutar algoritmo
        # Mostrar resultados
        ...
```

4. Recomendaciones para la Documentación

4.1 Manual de Usuario

Estructura recomendada:

1. **Introducción**
    - Propósito de la aplicación
    - Requisitos del sistema
2. **Instalación**
    - Pasos detallados
    - Configuración necesaria
3. **Guía de uso por módulo**
    - Descripción de cada pantalla
    - Funcionamiento de los controles
    - Ejemplos ilustrativos
4. **Ejemplos prácticos**
    - Casos de uso completos
    - Interpretación de resultados
5. **Solución de problemas comunes**
    - FAQs
    - Mensajes de error y su significado

4.2 Manual Técnico

Estructura recomendada:

1. **Arquitectura del sistema**
    - Diagrama general
    - Descripción de componentes
2. **Especificaciones técnicas**
    - Requisitos de hardware y software
    - Dependencias y bibliotecas
3. **Módulos implementados**
    - Descripción detallada de cada módulo
    - Algoritmos utilizados (con fórmulas matemáticas)
    - Diagramas de flujo
4. **Implementación**
    - Estructura de clases y funciones
    - Pruebas realizadas
    - Limitaciones conocidas
5. **Mantenimiento y extensión**
    - Guía para agregar nuevos métodos
    - Puntos de mejora futura

5. Puntos Importantes a Considerar

- **Validación de entradas**: Asegurar que los parámetros ingresados sean válidos
- **Manejo de errores**: Capturar y mostrar mensajes claros cuando algo falla
- **Visualización**: Utilizar gráficos apropiados para mostrar resultados

- **Eficiencia**: Optimizar algoritmos para casos con gran cantidad de datos
- **Testing**: Verificar cada método con problemas cuya solución es conocida

6. Recursos Recomendados

- **Libros de referencia**:
  - Chapra, S. C. (2007). Métodos Numéricos para Ingenieros (5ta ed.). McGraw-Hill.
  - Burden, Richard L. (2002). Análisis Numérico (7ma ed.). Paraninfo.
- **Documentación oficial**:
  - Python: https://docs.python.org/3/
  - NumPy: https://numpy.org/doc/
  - Matplotlib: https://matplotlib.org/stable/contents.html
  - SciPy: https://docs.scipy.org/doc/scipy/
- **Tutoriales en línea**:
  - Real Python: https://realpython.com/
  - Scipy Lecture Notes: https://scipy-lectures.org/

BIBLIOGRAFÍA

1. Chapra, S. C. (2007). Métodos Numéricos para Ingenieros (5ta ed.). McGraw-Hill.
2. Burden, Richard L. (2002). Análisis Numérico (7ma ed.). Paraninfo.
3. Lutz, Mark (2013). Learning Python (5ta ed.). O'Reilly Media.
4. Van der Walt, Stéfan, et al. (2011). "The NumPy array: A structure for efficient numerical computation". Computing in Science & Engineering.